📝 GLA UNIVERSITY — DAY 2 POST-ASSESSMENT QUESTION PAPER

Traditional Machine Learning & Parallel Computing using Intel oneAPI (`GLA-HPC-BCSG701`)

Time Allowed: 60 Minutes | **Total Questions:** 50 Multi-Choice Questions | **Maximum Marks:** 50
Candidate Name: _____ | **Roll Number:** _____
Date: _____ | **Section:** _____

Instructions to Candidates:

1. Each question carries 1 mark. There is no negative marking.
2. Select only ONE option (a, b, c, or d) for each question.
3. Carefully examine all mathematical derivations, OpenMP pragmas, MPI signatures, and oneDAL APIs before marking your answer.

Question 01 [Module 1: Parallel Computing Principles & Scaling Mathematics]

Question: An image preprocessing pipeline has a parallel fraction of 95% ($P = 0.95$) and a serial fraction of 5% ($S = 0.05$). What is the theoretical speedup when scaled to 64 CPU cores according to Amdahl's Law?

- **a)** 64.0x
- **b)** 15.42x
- **c)** 20.0x
- **d)** 32.0x

Candidate Selection: [ ]

Question 02 [Module 1: Parallel Computing Principles & Scaling Mathematics]

Question: Under Gustafson's Law for weak scaling, if the serial fraction of a scaled workload is 5% ($\alpha = 0.05$), what is the scaled speedup $S(P) = P - \alpha(P - 1)$ on 64 processors?

- **a)** 15.42x
- **b)** 60.85x
- **c)** 64.0x
- **d)** 3.2x

Candidate Selection: [ ]

Question 03 [Module 1: Parallel Computing Principles & Scaling Mathematics]

Question: In empirical benchmarks, an algorithm running on 8 CPU cores achieves a speedup of 9.2x (greater than 8x). What is the primary physical hardware explanation for this 'Superlinear Speedup'?

- **a)** The CPU frequency spontaneously tripled due to quantum resonance
- **b)** Cache Aggregation: Partitioning the dataset across 8 cores reduced each core's working set size such that it fit entirely within fast L2/L3 on-chip caches, eliminating slow DDR main memory accesses
- **c)** The operating system disabled interrupt processing permanently
- **d)** The C++ compiler skipped executing half of the loop iterations

Candidate Selection: [ ]

Question 04 [Module 1: Parallel Computing Principles & Scaling Mathematics]

Question: What does the Karp-Flatt metric $e = \frac{1/S(P) - 1/P}{1 - 1/P}$ measure when analyzing parallel scaling across increasing processor counts?

- **a)** The electrical power consumption of the CPU socket in Watts
- **b)** The experimentally observed serial fraction of the parallel application, revealing whether scaling degradation is caused by algorithmic serial bottlenecks or communication/synchronization overhead
- **c)** The percentage of network packet drop in InfiniBand fabrics
- **d)** The clock frequency jitter across NUMA nodes

Candidate Selection: [ ]

Question 05 [Module 1: Parallel Computing Principles & Scaling Mathematics]

Question: How does the Communication-to-Computation ratio (C/C) dictate the scalability of a distributed parallel algorithm as cluster node count increases?

- **a)** A high C/C ratio guarantees linear scaling to thousands of nodes
- **b)** As node count increases, if the communication overhead grows faster than local computation shrinks, the application encounters an efficiency cliff where adding more nodes slows down execution
- **c)** Communication overhead is always zero in MPI
- **d)** Computation time automatically increases when network switches are fast

Candidate Selection: [ ]

Question 06 [Module 1: Parallel Computing Principles & Scaling Mathematics]

Question: Which of the following is NOT an inherent source of parallel execution overhead in multi-threaded/distributed systems?

- **a)** Thread creation, scheduling, and context-switching overhead
- **b)** Synchronization barriers and lock contention latency
- **c)** Inter-process network message transfer latency
- **d)** Vector SIMD single-instruction mathematical throughput

Candidate Selection: [ ]

Question 07 [Module 1: Parallel Computing Principles & Scaling Mathematics]

Question: What distinguishes Uniform Memory Access (UMA) from Non-Uniform Memory Access (NUMA) multi-socket systems?

- **a)** UMA systems have no RAM; NUMA systems have infinite RAM
- **b)** In UMA, memory access latency is identical for all CPUs to all memory locations; in NUMA, access latency depends on whether memory is attached to the local CPU socket or a remote CPU socket across the interconnect bus
- **c)** NUMA only works on 32-bit operating systems
- **d)** UMA is only used for GPUs

Candidate Selection: [ ]

Question 08 [Module 1: Parallel Computing Principles & Scaling Mathematics]

Question: In parallel scalability theory, what does the 'Isoefficiency Function' quantify?

- **a)** The rate at which problem size W must grow with processor count P to maintain a constant parallel efficiency E
- **b)** The thermal efficiency of CPU fans
- **c)** The ratio of integer operations to floating-point operations
- **d)** The speed of the network router divided by RAM speed

Candidate Selection: [ ]

Question 09 [Module 1: Parallel Computing Principles & Scaling Mathematics]

Question: Why does running 64 threads on a 64-core Xeon server often produce zero speedup for a streaming memory operation like `y[i] = a * x[i] + b` (STREAM Triad)?

- **a)** Because OpenMP limits thread count to 8
- **b)** Because the workload is strictly memory-bandwidth bound, and a small number of cores (e.g., 8-12 cores) already completely saturate the physical DDR5 memory channels
- **c)** Because addition operations cannot be vectorized
- **d)** Because Linux CFS scheduler suspends threads that write memory

Candidate Selection: [ ]

Question 10 [Module 1: Parallel Computing Principles & Scaling Mathematics]

Question: Under strong scaling, what ultimately causes the speedup curve to bend downwards (negative speedup) at very high core counts?

- **a)** The problem size per core becomes so small that inter-thread communication, barrier synchronization, and serial tail latency dominate total runtime
- **b)** The compiler optimization level degrades automatically
- **c)** The CPU enters sleep mode due to lack of instructions
- **d)** The hard disk fills up with compiler logs

Candidate Selection: [ ]

Question 11 [Module 2: OpenMP for Multithreading]

Question: Why will `#pragma omp parallel for` produce incorrect mathematical results when applied to: `for (int i=1; i<N; i++) fib[i] = fib[i-1] + fib[i-2];`?

- **a)** Because OpenMP does not support the Fibonacci sequence
- **b)** Because the loop contains a true Read-After-Write (RAW) loop-carried dependency where each iteration requires values computed in immediately preceding iterations
- **c)** Because array indexing must start from 0 in parallel loops
- **d)** Because `fib` is a reserved keyword in C++

Candidate Selection: [ ]

Question 12 [Module 2: OpenMP for Multithreading]

Question: What is the critical behavioral difference between `private(x)` and `firstprivate(x)` in an OpenMP parallel region?

- **a)** `firstprivate(x)` initializes each thread's private copy of `x` with the value `x` had immediately prior to entering the parallel construct; `private(x)` leaves the private copy uninitialized
- **b)** `private(x)` allows all threads to write to the same memory location
- **c)** `firstprivate(x)` only creates a private variable for Thread 0
- **d)** `firstprivate(x)` allocates `x` on the GPU

Candidate Selection: [ ]

Question 13 [Module 2: OpenMP for Multithreading]

Question: What does `#pragma omp barrier` enforce within a parallel team of threads?

- **a)** It terminates all threads immediately
- **b)** It forces every thread in the team to halt at that execution point until all other threads in the team have arrived, ensuring memory consistency before proceeding
- **c)** It locks the motherboard memory bus for 1 second
- **d)** It prints the thread IDs to the console

Candidate Selection: [ ]

Question 14 [Module 2: OpenMP for Multithreading]

Question: When is it safe and advantageous to append the `nowait` clause to a work-sharing construct like `#pragma omp for nowait`?

- **a)** When the compiler gives an error during compilation
- **b)** When subsequent code does not depend on the results of the completed loop, allowing faster threads to proceed immediately to the next block without waiting at the implicit barrier
- **c)** When executing on a single-core CPU
- **d)** When all loop variables are declared global

Candidate Selection: [ ]

Question 15 [Module 2: OpenMP for Multithreading]

Question: What is the difference between `#pragma omp master` and `#pragma omp single` in OpenMP?

- **a)** `master` is executed exclusively by thread 0 with NO implicit barrier; `single` is executed by whichever thread arrives first and includes an implicit barrier at the end (unless `nowait` is specified)
- **b)** `single` executes on 1 core; `master` executes on all cores
- **c)** `master` is deprecated in OpenMP 5.0 and replaced by `python`
- **d)** `single` only executes once across the entire operating system boot

Candidate Selection: [ ]

Question 16 [Module 2: OpenMP for Multithreading]

Question: How does OpenMP `schedule(guided, chunk)` dynamically adjust chunk sizes across loop iterations?

- **a)** It randomly picks chunk sizes using a random number generator
- **b)** It starts by assigning large chunks of iterations to minimize scheduling overhead, and exponentially decreases chunk size down to `chunk` as remaining iterations diminish to balance the tail workload
- **c)** It divides all iterations equally among threads once at startup
- **d)** It assigns 1 iteration per thread strictly throughout the loop

Candidate Selection: [ ]

Question 17 [Module 2: OpenMP for Multithreading]

Question: What does setting `export OMP_PROC_BIND=close` versus `export OMP_PROC_BIND=spread` instruct the OpenMP runtime when placing threads across CPU sockets?

- **a)** `close` places threads consecutively on cores close to the master thread (sharing L3 cache); `spread` distributes threads evenly across separate sockets/NUMA nodes (maximizing memory bandwidth)
- **b)** `close` terminates threads when idle; `spread` duplicates threads
- **c)** `spread` spreads data across the network via Wi-Fi
- **d)** `close` limits execution to 1 thread

Candidate Selection: [ ]

Question 18 [Module 2: OpenMP for Multithreading]

Question: What is the benefit of adding `collapse(2)` to `#pragma omp parallel for` on nested loops: `for (int i=0; i<N; i++) for (int j=0; j<M; j++)`?

- **a)** It deletes the outer loop entirely
- **b)** It flattens the two nested loops into a single linear iteration space of size $N \times M$, creating a much larger pool of parallel work items to distribute across large core counts
- **c)** It reduces memory usage by 50%
- **d)** It forces both loops to execute backwards

Candidate Selection: [ ]

Question 19 [Module 2: OpenMP for Multithreading]

Question: Why is the OpenMP Tasking construct (`#pragma omp task`) ideal for parallelizing recursive algorithms (e.g., Quicksort or tree traversal) compared to standard `#pragma omp for` ?

- **a)** Because `#pragma omp for` can only run on arrays with prime lengths
- **b)** Because tasks can be spawned dynamically and placed into a runtime work pool executed asynchronously by any available thread, enabling irregular and recursive parallelism
- **c)** Because tasks execute in the kernel space without memory stacks
- **d)** Because tasks disable compiler warnings

Candidate Selection: [ ]

Question 20 [Module 2: OpenMP for Multithreading]

Question: Why do senior HPC engineers enforce `#pragma omp parallel default(none)` in all production C/C++ OpenMP codebases?

- **a)** To disable all compiler optimizations
- **b)** It forces the programmer to explicitly declare the data-sharing attribute (`shared` , `private` , etc.) of every single variable used in the construct, catching accidental race conditions at compile time
- **c)** To prevent the code from compiling on AMD processors
- **d)** To force OpenMP to run sequentially

Candidate Selection: [ ]

Question 21 [Module 3: MPI for Distributed Systems]

Question: What is the primary function of `MPI_Comm_split(comm, color, key, &newcomm)` in high-performance computing clusters?

- **a)** To divide the cluster electrical power among nodes
- **b)** To partition an existing communicator into disjoint sub-communicators based on a `color` parameter, enabling localized group collective communication (e.g., node-local communicators)
- **c)** To terminate half of the MPI processes
- **d)** To split a single file into two pieces on the hard drive

Candidate Selection: [ ]

Question 22 [Module 3: MPI for Distributed Systems]

Question: What is the key advantage of non-blocking communication (`MPI_Isend` , `MPI_Irecv`) paired with `MPI_Wait()` in high-concurrency scientific simulations?

- **a)** It doubles the network wire speed
- **b)** It initiates message transfer asynchronously and returns immediately, allowing the CPU to compute interior domain cells while network hardware transfers boundary data in parallel (computation-communication overlap)
- **c)** It eliminates the need for network switches
- **d)** It converts MPI messages into encrypted emails

Candidate Selection: [ ]

Question 23 [Module 3: MPI for Distributed Systems]

Question: What distinguishes Synchronous Send (`MPI_Ssend`) from Standard Send (`MPI_Send`) in the MPI standard?

- **a)** `MPI_Ssend` is 100 times faster than `MPI_Send`
- **b)** `MPI_Ssend` guarantees that the send operation will NOT complete until the matching receive has been posted and has started receiving data (rendezvous handshake); standard `MPI_Send` may buffer and return early
- **c)** `MPI_Ssend` only operates on single-byte characters
- **d)** `MPI_Ssend` sends messages through hard disk files

Candidate Selection: [ ]

Question 24 [Module 3: MPI for Distributed Systems]

Question: In an MPI cluster of P ranks, what data exchange pattern is performed by `MPI_Alltoall`?

- **a)** Rank 0 sends data to Rank 1 only
- **b)** Every rank sends a distinct chunk of data to every other rank in the communicator (a complete distributed matrix transpose / all-to-all personalized exchange)
- **c)** Every rank sends data to the internet
- **d)** All ranks delete their local data arrays

Candidate Selection: [ ]

Question 25 [Module 3: MPI for Distributed Systems]

Question: Why does using `MPI_Sendrecv` prevent the classic deadlock that occurs when adjacent processes attempt simultaneous blocking `MPI_Send` and `MPI_Recv` operations?

- **a)** It disables network error checking
- **b)** It executes the send and receive operations atomically under an internally managed non-blocking protocol, guaranteeing that communication proceeds without circular buffer dependency deadlocks
- **c)** It forces one process to terminate itself
- **d)** It writes both messages to a shared SQLite database

Candidate Selection: [ ]

Question 26 [Module 3: MPI for Distributed Systems]

Question: Why would an HPC developer create an `MPI_Type_vector` datatype instead of copying matrix elements into a temporary contiguous buffer before calling `MPI_Send`?

- **a)** To allow sending data to non-MPI programs
- **b)** To enable zero-copy transfers of non-contiguous strided memory blocks (such as a matrix column in row-major layout) directly by the network hardware/driver, eliminating memory packing/unpacking overhead
- **c)** Because MPI does not support loops
- **d)** To encrypt data across sockets

Candidate Selection: [ ]

Question 27 [Module 3: MPI for Distributed Systems]

Question: On a cluster with P nodes, what is the theoretical communication time complexity of an optimized `MPI_Bcast` implemented via a binomial spanning tree compared to a naive root loop?

- **a)** Binomial tree takes $O(\log_2 P)$ steps; naive loop takes $O(P)$ steps
- **b)** Binomial tree takes $O(P^2)$ steps; naive loop takes $O(1)$ steps
- **c)** Both take $O(P)$ steps
- **d)** Binomial tree takes infinite steps

Candidate Selection: [ ]

Question 28 [Module 3: MPI for Distributed Systems]

Question: What potential hazard arises when using `MPI_ANY_SOURCE` and `MPI_ANY_TAG` wildcards in high-throughput MPI message reception loops?

- **a)** The computer loses internet connection
- **b)** Non-deterministic message matching: messages from different sender ranks may arrive out of order, leading to non-reproducible race conditions or incorrect data association if matching logic is flawed
- **c)** The MPI library crashes with a syntax error
- **d)** The CPU frequency drops to 1 GHz

Candidate Selection: [ ]

Question 29 [Module 3: MPI for Distributed Systems]

Question: What is the primary benefit of creating a virtual Cartesian grid topology via `MPI_Cart_create` in 2D/3D numerical physics simulations?

- **a)** It enables 3D graphics rendering on the cluster display
- **b)** It maps 1D linear MPI ranks onto logical multi-dimensional coordinates and provides `MPI_Cart_shift` to automatically identify neighboring rank IDs in North, South, East, and West directions
- **c)** It forces all compute nodes to be placed in the same room
- **d)** It doubles the RAM available to each process

Candidate Selection: [ ]

Question 30 [Module 3: MPI for Distributed Systems]

Question: What defines MPI-2/MPI-3 One-Sided Communication (Remote Memory Access / RMA) through `MPI_Put` and `MPI_Get`?

- **a)** Only one computer is turned on during execution
- **b)** A process can directly read from or write to the memory window of a remote process without requiring the remote process to explicitly participate in matching receive or send calls
- **c)** Messages can only be sent once per day

- **d)** RMA only works over serial cables

Candidate Selection: [ ]

Question 31 [Module 4: Data Decomposition & Synchronization]

Question: In the MESI cache coherence protocol, what happens when Core 1 writes to a 64-byte cache line currently held in the 'Shared' (S) state by Core 0 and Core 1?

- **a)** Core 1 sends an Invalidating Bus Transaction; Core 0 transitions its copy to 'Invalid' (I), while Core 1 updates the data and transitions its line to 'Modified' (M)
- **b)** Both cores freeze execution and reboot
- **c)** Core 0 overwrites Core 1's write immediately
- **d)** The cache line is permanently deleted from RAM

Candidate Selection: [ ]

Question 32 [Module 4: Data Decomposition & Synchronization]

Question: How can a C++ developer permanently eliminate False Sharing when declaring per-thread state structures in OpenMP?

- **a)** By declaring all variables as `static`
- **b)** By using the C++11 `alignas(64)` specifier (or manual byte padding) to ensure each thread's independent structure starts on its own unique 64-byte cache line boundary
- **c)** By turning off CPU turbo boost in the BIOS
- **d)** By using 32-bit integers instead of 64-bit integers

Candidate Selection: [ ]

Question 33 [Module 4: Data Decomposition & Synchronization]

Question: In a distributed PDE solver using a 4th-order spatial central difference stencil that accesses 2 neighbors in each direction ($i-2, i-1, i, i+1, i+2$), what is the minimum required Halo (Ghost) cell layer thickness?

- **a)** 1 layer of cells
- **b)** 2 layers of cells
- **c)** 4 layers of cells
- **d)** 8 layers of cells

Candidate Selection: [ ]

Question 34 [Module 4: Data Decomposition & Synchronization]

Question: For a 3D grid simulation of size $N \times N \times N$ partitioned among P cluster processors, how does the communication surface area scale under 3D cube decomposition compared to 1D slab decomposition?

- **a)** 3D cube decomposition exchanges $O(N^2/P^{2/3})$ elements per node, which is dramatically smaller than 1D slab decomposition exchanging $O(N^2)$ elements per node at large P
- **b)** 1D slab decomposition has zero communication

- **c)** 3D decomposition requires 3 times more communication than 1D
- **d)** Both decompositions have identical communication volumes

Candidate Selection: [ ]

Question 35 [Module 4: Data Decomposition & Synchronization]

Question: Which compiler instrumentation flag can be passed to `gcc`, `clang`, or `icpx` to detect data races at runtime in multi-threaded C/C++ applications?

- **a)** `-O3`
- **b)** `-fsanitize=thread`
- **c)** `-fno-exceptions`
- **d)** `-Wall`

Candidate Selection: [ ]

Question 36 [Module 4: Data Decomposition & Synchronization]

Question: How does the 'Work Stealing' scheduling algorithm handle load imbalance in parallel task runtimes (e.g., Cilk, Intel TBB)?

- **a)** It forces all processors to execute identical numbers of instructions
- **b)** Each processor maintains a local double-ended queue (deque) of tasks; when a processor runs out of work, it attempts to steal tasks from the tail of another processor's deque
- **c)** It terminates slow tasks and restarts them on the master thread
- **d)** It pauses all processors until the slowest processor finishes

Candidate Selection: [ ]

Question 37 [Module 4: Data Decomposition & Synchronization]

Question: What fundamental CPU atomic instruction forms the basis of lock-free concurrent queues and stacks on modern x86_64 and ARM processors?

- **a)** Bitwise Shift Left (`SHL`)
- **b)** Compare-And-Swap (`CMPXCHG` on x86, `CAS` or `LDREX/STREX` on ARM)
- **c)** Floating-point Square Root (`FSQRT`)
- **d)** No-Operation (`NOP`)

Candidate Selection: [ ]

Question 38 [Module 4: Data Decomposition & Synchronization]

Question: When `malloc()` allocates a 50 GB array on a dual-socket Linux NUMA server, which physical socket's RAM controller is the memory allocated from?

- **a)** Socket 0 strictly, regardless of which thread touches it

- **b)** Neither socket at `malloc()` time; physical memory pages are only mapped when a thread FIRST writes to a page, allocating the page from the local NUMA memory node of the touching thread (First-Touch Policy)
- **c)** Memory is evenly distributed across both sockets at `malloc()` time
- **d)** Memory is allocated in the CPU L1 cache

Candidate Selection: [ ]

Question 39 [Module 4: Data Decomposition & Synchronization]

Question: What is the 'ABA Problem' in concurrent lock-free data structures using Compare-And-Swap (CAS)?

- **a)** An error when compiling code on ARM architecture
- **b)** A hazard where a memory location holds value A, changes to B, and changes back to A; a CAS operation succeeds assuming nothing changed, while intermediate pointer structures have been invalidated or reallocated
- **c)** When three threads run in alphabetical order
- **d)** A failure in the CPU floating-point unit

Candidate Selection: [ ]

Question 40 [Module 4: Data Decomposition & Synchronization]

Question: Which memory consistency model is enforced by standard x86_64 hardware architecture for memory loads and stores?

- **a)** Total Store Order (TSO), where stores are not reordered with other stores, and loads are not reordered with other loads, but a store followed by a load may be reordered
- **b)** Strict Sequential Consistency for all operations with zero buffering
- **c)** Completely Relaxed Memory Model (ARM Weak Ordering)
- **d)** Random instruction reordering without store buffers

Candidate Selection: [ ]

Question 41 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: What is the foundational architectural advantage of Intel oneDAL over generic open-source Scikit-Learn backends?

- **a)** oneDAL converts Python scripts into JavaScript
- **b)** oneDAL implements classical algorithms in optimized C++ utilizing multi-threading, CPU cache blocking, vectorization (AVX-512), and hardware matrix engines (Intel AMX) on modern Xeon processors
- **c)** oneDAL removes all math operations to run faster
- **d)** oneDAL requires zero electricity to run

Candidate Selection: [ ]

Question 42 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: If a developer wants to patch ONLY the K-Means and Support Vector Classifier (SVC) algorithms while keeping all other Scikit-Learn estimators unpatched, how can this be achieved?

- **a)** `patch_sklearn(['kmeans', 'svc'])`
- **b)** `patch_sklearn(name='kmeans', name2='svc')`
- **c)** `patch_sklearn(global=False)`
- **d)** Selective patching is impossible; `patch_sklearn()` always patches everything

Candidate Selection: [ ]

Question 43 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: How does Intel oneDAL optimize the Lloyd's algorithm iteration step in K-Means clustering for large datasets (e.g., 500,000 samples $\times$ 50 features)?

- **a)** By using the Triangle Inequality to filter out centroid distance calculations that cannot possibly change cluster assignment, and computing batch distances using GEMM matrix multiplication
- **b)** By assigning every point to cluster 0
- **c)** By running K-Means on only 10 data points
- **d)** By converting all numbers to 8-bit integers with loss of precision

Candidate Selection: [ ]

Question 44 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: What algorithmic innovation does oneDAL implement to achieve up to 20x faster inference for Random Forest classification?

- **a)** It deletes all decision trees except one
- **b)** It utilizes vectorized tree traversal, evaluating multiple data samples or multiple tree nodes simultaneously using SIMD vector registers rather than serial scalar pointer chasing
- **c)** It converts decision trees into neural networks
- **d)** It runs predictions on the hard drive

Candidate Selection: [ ]

Question 45 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: When executing `np.dot(A, B)` on large matrices in an environment with the Intel AI Analytics Toolkit, why is performance substantially faster than default pip NumPy?

- **a)** Because Intel NumPy rounds numbers to the nearest integer
- **b)** Because it links directly against Intel oneMKL (Math Kernel Library), which dynamically dispatches multi-threaded AVX-512 and AMX assembly routines optimized for Xeon cache hierarchies
- **c)** Because it skips calculating the off-diagonal elements
- **d)** Because it runs in the Linux kernel space

Candidate Selection: [ ]

Question 46 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: What code modification is required in an existing Python script to accelerate Pandas data processing across all 64 cores of an HPC node using Modin?

- **a)** Rewrite all dataframe operations using SQL queries
- **b)** Replace `import pandas as pd` with `import modin.pandas as pd`
- **c)** Add `with multiprocessing.Pool(64):` around every line of Pandas code
- **d)** Convert all dataframes into CSV files on disk

Candidate Selection: [ ]

Question 47 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: Why does training a non-linear SVM (RBF kernel) on large datasets scale poorly in standard Scikit-Learn ($O(N^2)$ to $O(N^3)$), and how does oneDAL mitigate this bottleneck?

- **a)** Standard Scikit-learn cannot compute exponential functions; oneDAL approximates all kernels as linear
- **b)** Kernel SVM requires computing an $N \times N$ dense kernel matrix; oneDAL uses efficient block-kernel caching, vectorized matrix evaluations, and parallelized working set selection in the SMO solver
- **c)** oneDAL limits SVM training to a maximum of 100 samples
- **d)** oneDAL runs SVM on the cluster network router

Candidate Selection: [ ]

Question 48 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: How can an engineer direct an Intel-accelerated Scikit-learn estimator (e.g., `KMeans`) to execute on an Intel Data Center GPU Max rather than the host Xeon CPU?

- **a)** By using the `dpctl` device context: `with dpctl.device_context('gpu'): kmeans.fit(X)`
- **b)** By setting the environment variable `USE_GPU=1`
- **c)** By renaming the Python script to `gpu_script.py`
- **d)** By disconnecting the CPU power supply

Candidate Selection: [ ]

Question 49 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: In high-dimensional feature spaces ($100,000 \times 1,000$), which core mathematical operation in PCA is accelerated by Intel oneDAL?

- **a)** Singular Value Decomposition (SVD) and symmetric eigenvalue decomposition of the covariance matrix ($X^T X$)
- **b)** Sorting rows alphabetically by feature name
- **c)** Random Gaussian noise injection
- **d)** Huffman coding compression

Candidate Selection: [ ]

Question 50 [Module 5: Intel AI Analytics Toolkit & oneDAL Acceleration]

Question: How can a developer programmatically verify inside Python that Scikit-Learn has been successfully patched by `scikit-learn-intelex`?

- **a)** By printing `sklearn.cluster.KMeans.__module__` and checking that it points to `daal4py.sklearn` or `sklearnex`
- **b)** By checking if the computer's cooling fan is running
- **c)** By verifying that the script executes in exactly 0.000 seconds
- **d)** By checking the operating system version string

Candidate Selection: [ ]
